Blockchain-Based Supply Chain Financing Platform - Group 13

1. **Environment**:
    - Solidity version: **0.8.20**
    - Platform: **Remix VM (Cancun)**
    - Accounts used:
        - Account 1 (Verifier/Admin)
        - Account 2 (Exporter)
        - Account 3 (Lender)

2. **Test Case 1**
   Register Document
    - Action: Account 2 calls registerDocument("INV001", 1 ether)
    - Expected Result:
        - A new document is created
        - documentId = 1
        - status = Registered (0)
    - Evidence: Figure 1 (DocumentRegistered event)

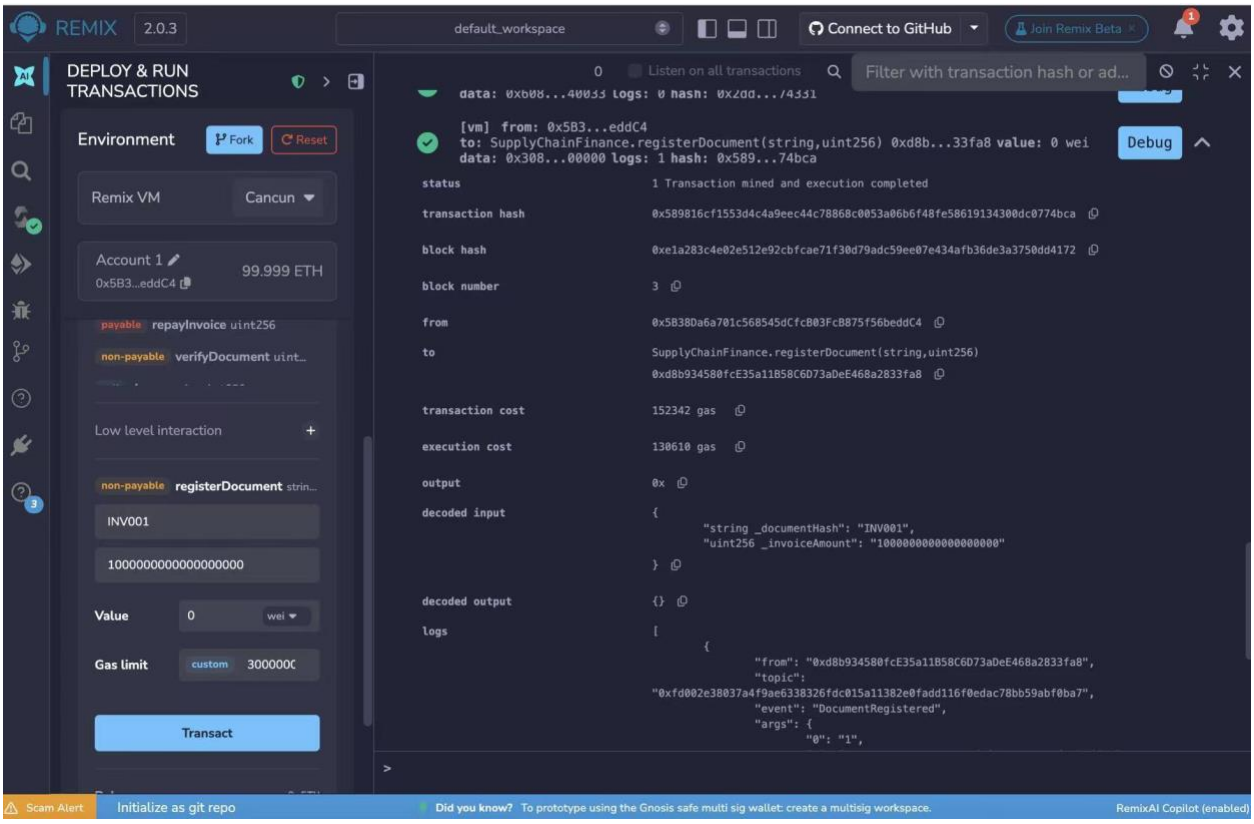

Figure 1: Register Document

3. **Test Case 2**

Verify Document

- Action: Account 1 calls verifyDocument(1)
- Expected Result:
  - Document status changes from Registered (0) to Verified (1)
- Evidence: Figure 2 (DocumentVerified event)

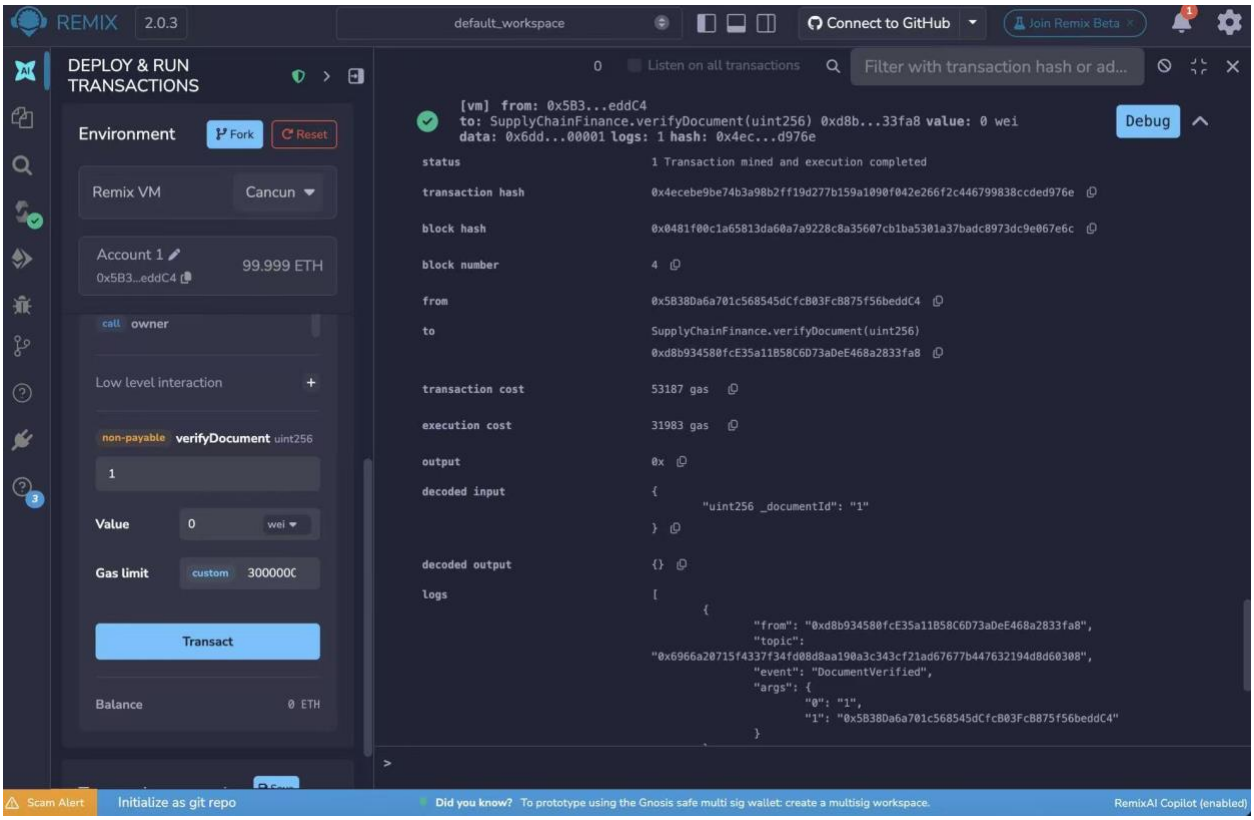

Figure 2: Verified Document

4. **Test Case 3**

Fund Invoice

- Action: Account 3 calls fundInvoice(1) with 1 ether
- Expected Result:
  - Document status changes to Funded (2)
  - Lender address is recorded
  - Funds are transferred to exporter
- Evidence: Figure 3 (InvoiceFunded event)

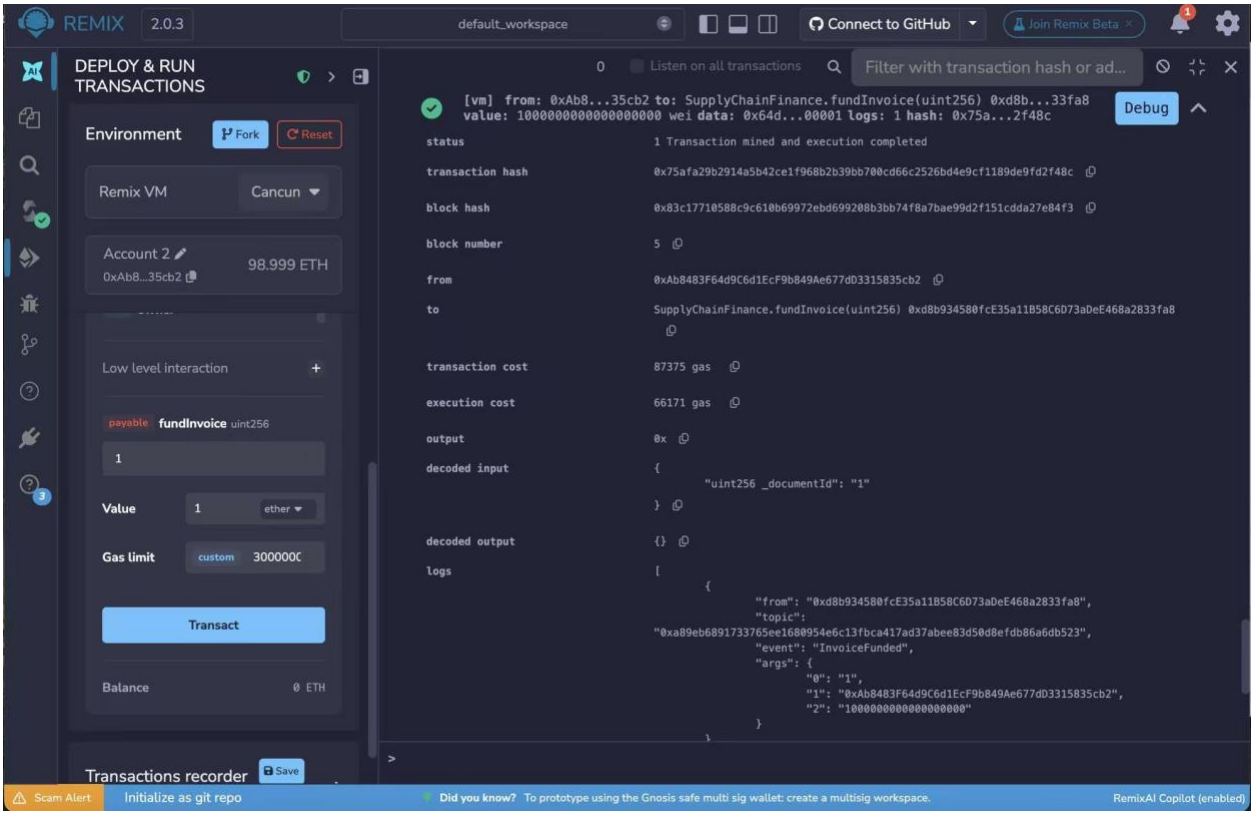

Figure 3: Fund Invoice

5. **Test Case 4**

   Repay Invoice

   - Action: Account 2 calls repayInvoice(1) with 1 ether
   - Expected Result:
     - Document status changes to Repaid (3)
     - Lender receives repayment
   - Evidence: Figure 4 (InvoiceRepaid event)

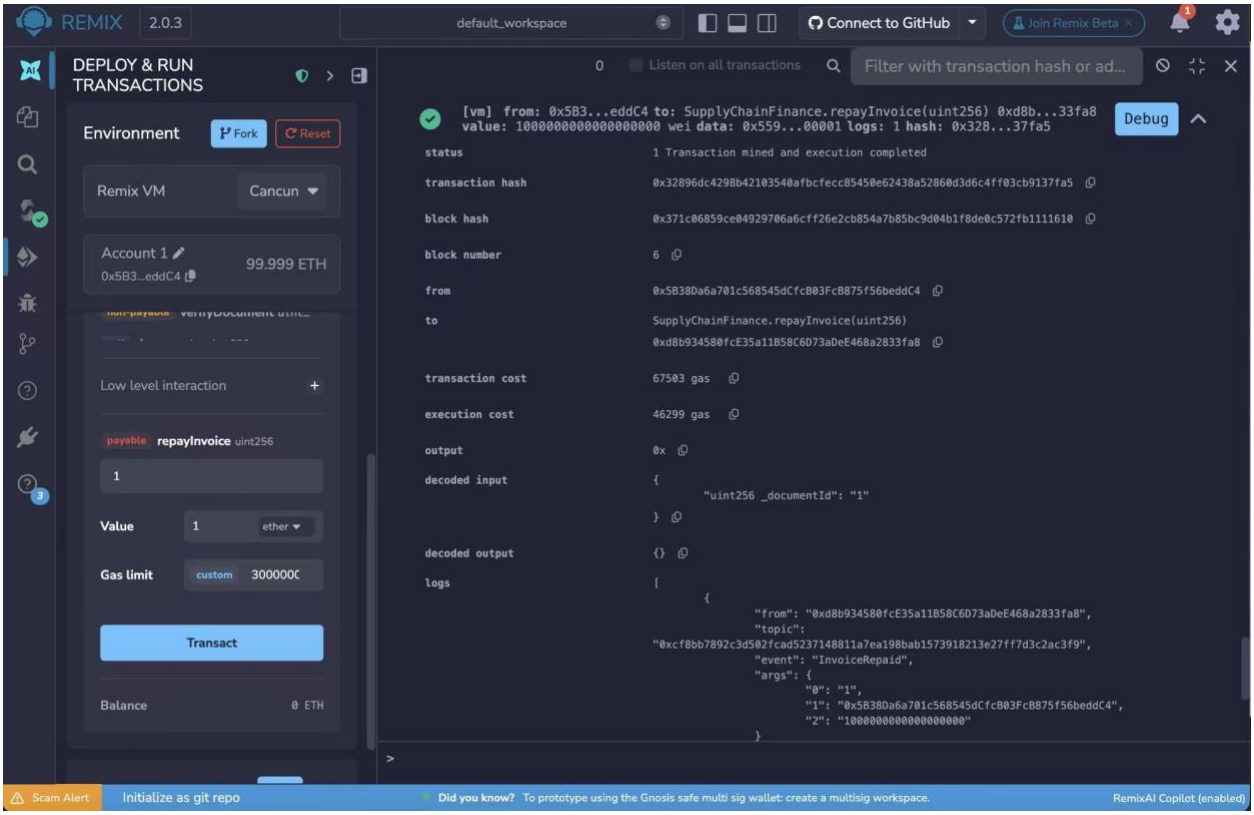

Figure 4: Repay Invoice

6. **Test Case 5**

Retrieve Document Data

- Action: Call getDocument(1)
- Expected Result:
  - Returns correct document details:
    - documentId = 1
    - documentHash = "INV001"
    - exporter, verifier, lender addresses
    - invoiceAmount, fundedAmount, repaidAmount
    - status = Repaid
- Evidence: Figure 5 (Decoded output)

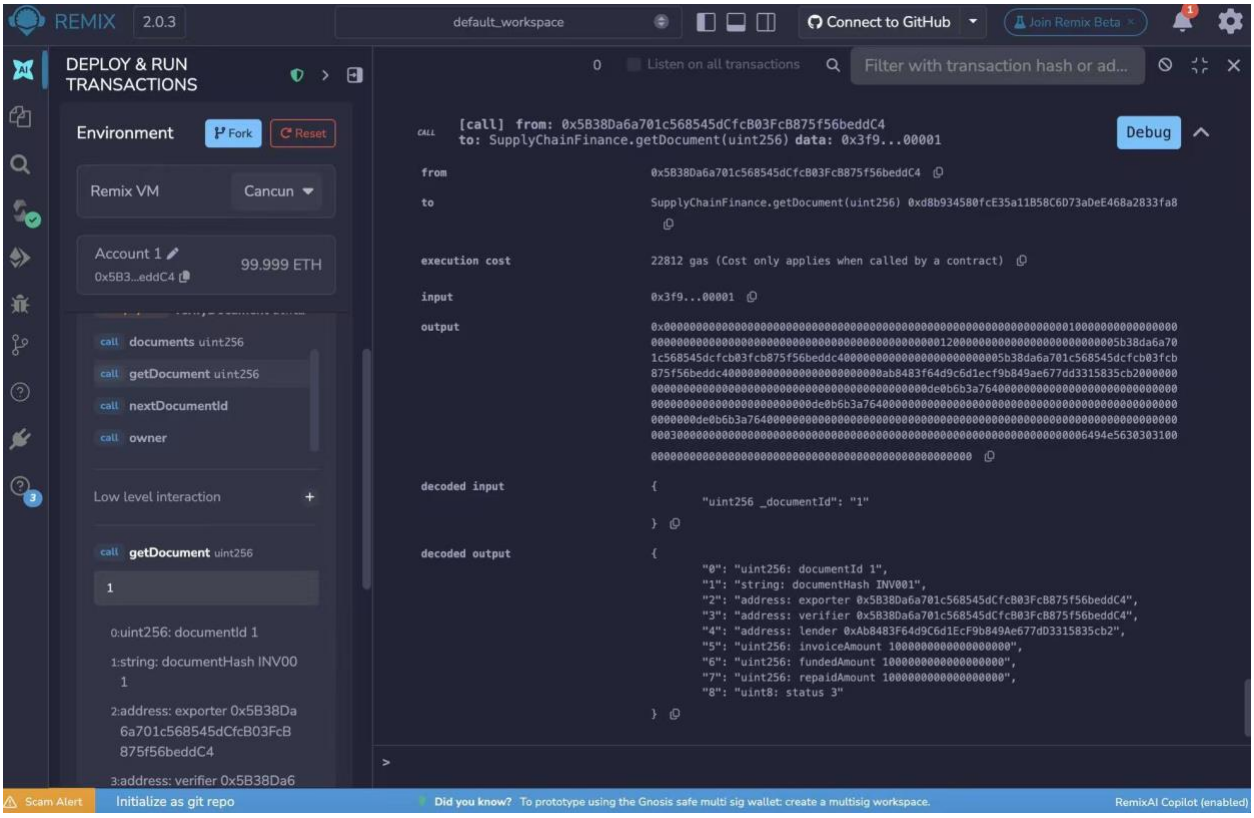

Figure 5: Retrieve Document Data

7. **Test Case 6**

Prevent Funding Before Verification

- Action: Try to call fundInvoice() before verifyDocument()
- Expected Result:
  - Transaction fails (invalid state)

8. **Test Case 7**

Prevent Duplicate Funding

- Action: Try to fund the same invoice again after it is already funded
- Expected Result:
  - Transaction fails

Remix is only the development and demonstration environment used in our class prototype. The actual architecture we are testing is an Ethereum-compatible smart contract workflow. In production, banks, auditors, logistics verifiers, and financing platforms would not manually interact through Remix. Instead, they would connect to the smart contract through backend systems, APIs, or institutional wallets.

We use Ethereum because trade finance requires a shared state layer across parties that do not fully trust one centralized database. A traditional database can record invoice status, but it is controlled by one institution. An Ethereum smart contract creates a neutral, tamper-resistant workflow where invoice registration, verification, funding, and repayment are recorded as transparent state transitions.

In our prototype, Account 1 represents the verifier or platform administrator, Account 2 represents the exporter, and Account 3 represents the lender. A production version could replace the single verifier with a consortium of approved banks, auditors, logistics verifiers, or trusted agencies. These institutions could attest to off-chain verified invoice hashes, while the smart contract stores the verified hash and financing status on-chain.

Therefore, Ethereum is not used to replace banks or legal verification. It is used to coordinate them through a shared, programmable, and auditable state machine.